Stack Android simple pour APK

Version allégée pour logiciel de caisse restaurant

Note technique courte — 3 juin 2026

Recommandation
Pour le client actuel et un devis limité, partir sur une **application Android native en APK** avec **Kotlin**, **Jetpack Compose**, **Room / SQLite** et une impression validée sur le matériel réel. Cette stack est plus simple à livrer qu'une architecture Electron ou cloud, et correspond mieux à une tablette ou un terminal POS Android.

1. Objectif

Livrer une première version exploitable centrée sur l'encaissement local :

— installer l'application sous forme de fichier `.apk` ;
— fonctionner sans connexion Internet ;
— stocker les articles, commandes, paiements et historiques localement ;
— imprimer une note client après validation du matériel ;
— produire des exports simples en `JSON` ou `CSV`.

Cette proposition sert au périmètre client court. L'option interne Electron / React peut rester une piste d'apprentissage ou de produit futur.

2. Stack proposée

Besoin	Choix	Rôle
Plateforme cible	Android	Installer l'application sur tablette Android ou terminal POS Android.
Format de livraison	`APK`	Fournir un fichier installable directement sur l'appareil du client.
Langage principal	Kotlin	Développer une application Android native maintenable.
Interface tactile	Jetpack Compose	Créer des écrans tactiles modernes avec peu de code boilerplate.
Base locale	Room / SQLite	Enregistrer articles, commandes, lignes, paiements et historiques localement.
Configuration simple	DataStore	Stocker les paramètres légers : nom du restaurant, préférences, imprimante.
Tickets et sauvegardes	JSON	Figer le contenu d'un ticket et exporter une sauvegarde lisible.
Exports tableur	CSV	Fournir un format simple à ouvrir pour le responsable ou le comptable.
Impression	SDK fabricant ou ESC/POS	Imprimer sur imprimante intégrée, Bluetooth, USB ou réseau après test réel.
Build	Android Studio / Gradle	Compiler et générer l'APK.
Tests	JUnit	Tester les calculs de panier, TVA, paiement mixte et rendu monnaie.
Versioning	Git	Suivre les changements et sécuriser les livraisons.

Règle métier importante : les montants sont stockés en centimes sous forme d'entiers, par exemple `1850` pour 18,50 euros.

3. Pourquoi cette stack est plus simple

Critère	APK Android native	Electron / React interne
Installation client	Un APK à installer sur l'appareil	Application desktop Windows à empaqueter
Matériel cible	Tablette ou terminal Android POS	Terminal Windows ou PC tactile
Nombre de briques	Kotlin, Compose, Room	Electron, React, TypeScript, Vite, SQLite
Coût matériel possible	Plus bas avec tablette Android simple	Souvent plus élevé sur tablette ou POS Windows
Logique offline	Naturelle avec base locale sur l'appareil	Possible aussi, mais architecture plus riche

4. Périmètre fonctionnel simple

— catalogue d'articles et catégories ;

— panier et type de commande : sur place, à emporter, livraison ;

— paiements : espèces, carte confirmée manuellement, titres-restaurant, autre ;

— paiement mixte et rendu monnaie ;
— génération d'une note client ;
— historique des encaissements ;
— annulation ou correction traçable ;
— statistiques journalières simples ;
— export `JSON` et `CSV`.

5. Points à valider avant chiffrage

— Le client possède-t-il déjà une tablette Android ou un terminal POS Android ?
— L'imprimante est-elle intégrée au terminal ou externe ?
— L'impression doit-elle passer par Bluetooth, USB, réseau ou SDK fabricant ?
— Le TPE bancaire reste-t-il autonome avec confirmation manuelle dans l'application ?
— L'installation de l'APK hors Play Store est-elle acceptée sur le matériel retenu ?

Point critique
Le vrai risque de cette stack n'est pas Kotlin : c'est l'impression. Le choix final du terminal doit donc être validé avec un test d'impression réel avant de promettre une livraison stable.

6. Plan d'action court

1. Créer le projet Android Kotlin dans Android Studio.
2. Construire les premiers écrans Compose : accueil, catalogue, panier.
3. Ajouter Room / SQLite : articles, commandes, lignes et paiements.
4. Implémenter les calculs : total TTC, paiements mixtes, rendu monnaie.
5. Générer un ticket JSON figé à chaque encaissement.
6. Ajouter historique, correction et annulation traçable.
7. Ajouter exports JSON et CSV.
8. Tester l'impression sur le terminal ou l'imprimante retenue.
9. Générer l'APK et tester l'installation sur l'appareil client.
10. Réaliser une recette courte avec quelques scénarios d'encaissement.

7. Sources officielles

— Android Kotlin et Jetpack Compose : documentation Android
— Jetpack Compose : documentation Android
— Room / SQLite : documentation Android
— DataStore : documentation Android

Document de travail. Cette stack vise une version plus simple et moins coûteuse pour un client précis. Les choix d'impression, de terminal et d'installation doivent être validés sur matériel réel.